

Real Python FastAPI Cheat Sheet

Practice at realpython.com with hands-on coding exercises, quizzes, and a personal AI mentor.

Install and Run

- Install inside a virtual environment
- `[standard]` adds the `fastapi` CLI and Uvicorn
- `dev` auto-reloads on save; `run` is for production

Install FastAPI

```
$ python -m pip install "fastapi[standard]"
```

Create a Minimal App (main.py)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello, FastAPI!"}
```

Start the Server

```
$ fastapi dev main.py
$ fastapi run main.py
$ uvicorn main:app --reload --port 8000
```

Learn More on realpython.com/search:

`fastapi` · `virtual environment`

Routes and HTTP Methods

- Returned dicts, lists, and models become JSON
- Use `async def` for I/O-bound work

Map Methods to Operations

```
@app.get("/books")          # Read all
@app.post("/books")         # Create
@app.get("/books/{id}")     # Read one
@app.put("/books/{id}")     # Replace
@app.patch("/books/{id}")   # Update
@app.delete("/books/{id}") # Delete
```

Learn More on realpython.com/search:

`crud` · `rest api` · `async`

Path and Query Parameters

- `{name}` in the path makes a path parameter
- Other args are query parameters; defaults make them optional
- Bad types return `422` automatically
- Declare fixed paths like `/books/new` before `/books/{book_id}`

Path Parameter

```
@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"id": book_id}

# GET /books/42  -> {"id": 42}
# GET /books/abc -> 422
```

Query Parameters

```
@app.get("/books")
def list_books(
    limit: int = 10,
    q: str | None = None,
):
    return {"limit": limit, "q": q}

# GET /books?limit=5&q=py
# -> {"limit": 5, "q": "py"}
```

Add Constraints

```
from typing import Annotated
from fastapi import Query

Limit = Annotated[int, Query(ge=1, le=100)]

@app.get("/items")
def items(limit: Limit = 10):
    return {"limit": limit}

# GET /items?limit=0 -> 422
```

Learn More on realpython.com/search:

`fastapi` · `type hints`

Request Bodies

- A Pydantic model argument reads the JSON body
- `Field()` adds validation and docs
- Invalid bodies get a `422` before your code runs
- Mix path, query, and body parameters freely

Define a Model

```
from pydantic import BaseModel, Field

class Book(BaseModel):
    title: str = Field(min_length=1)
    pages: int = Field(gt=0)
    author: str | None = None
```

Accept a JSON Body

```
@app.post("/books")
def create_book(book: Book):
    return book

# {"title": "Python Basics", "pages": 635}
# -> {..., "author": null}
```

Learn More on realpython.com/search:

`pydantic` · `json`

Responses and Errors

- `response_model` filters out extra fields
- A return type hint works the same way
- `status.HTTP_201_CREATED` names the code `201`
- `raise` an `HTTPException`, don't `return` it

Filter the Output

```
class BookOut(BaseModel):
    id: int
    title: str

@app.post(
    "/books",
    response_model=BookOut,
    status_code=201,
)
def create_book(book: Book):
    return {"id": 1, **book.model_dump()}
# 201 {"id": 1, "title": "..."}
```

Raise an HTTP Error

```
from fastapi import HTTPException

@app.get("/books/{book_id}")
def get_book(book_id: int):
    if book_id not in books:
        raise HTTPException(
            status_code=404,
            detail="Book not found",
        )
    return books[book_id]

# 404 {"detail": "Book not found"}
```

Learn More on realpython.com/search:

`http status codes` · `exceptions`

Dependencies

- `Depends()` runs a function before the endpoint
- Code after `yield` runs after the response
- Dependencies can take their own parameters

Share Query Parameters

```
from fastapi import Depends

def paginate(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
Page = Annotated[dict, Depends(paginate)]

@app.get("/books")
def list_books(page: Page):
    return page
```

Open and Close a Resource

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users")
def users(db=Depends(get_db)):
    return db.query(User).all()
```

Learn More on realpython.com/search:

`dependency injection` · `sqlalchemy`

Interactive Docs

- Built from your type hints; *Try it out* sends real requests

URL	Shows
`/docs`	Swagger UI playground
`/redoc`	ReDoc reference
`/openapi.json`	Raw OpenAPI schema

Name Your API

```
app = FastAPI(title="Books", version="1.0")
```

Learn More on realpython.com/search:

`openapi` · `fastapi`

Testing With TestClient

- `TestClient` calls your app without a server
- Run the tests with `pytest`

Test an Endpoint (test_main.py)

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_create_book():
    response = client.post(
        "/books",
        json={"title": "Py", "pages": 1},
    )
    assert response.status_code == 201
    assert response.json()["title"] == "Py"
```

Swap In a Fake Dependency

```
def fake_db():
    return FakeSession()

app.dependency_overrides[get_db] = fake_db
```

Learn More on realpython.com/search:

`pytest` · `testing`

Become a Python and AI expert with your free personal learning plan at realpython.com/learning-plan

Real Python